UT1 · PSP · 2º DAM

Programación multiproceso

Guía de estudio de tu zip: para qué sirve cada proyecto, cada bloque de código y cada método.

Cómo leer esta guía

AZUL · concepto
Azul: qué es y para qué sirve. Es la idea, sin código.

VERDE · bloque de código
Verde: un trozo de código explicado paso a paso, en bloques.

NARANJA · ojo
Naranja: errores típicos y trampas. Léelos dos veces.

MORADO · examen
Morado: lo que suelen preguntar en un examen.

Qué hay en el zip (en el orden en que conviene estudiarlo)

Proyecto	Qué practicas
1 · LectorProceso	Lanzar un comando y leer lo que escribe
2 · RedireccionProceso	Mandar la salida a pantalla o a fichero
3 · Ejercicio1	ping guardado en fichero + código de salida
4 · ControlCicloVidaProceso	Esperar con tiempo límite y matar procesos
5 · InteraccionProceso	Mandar datos al hijo (su entrada)
6 · Diagnóstico de red	ping según el SO + log con cabecera
7 · Lanzador y Registrador	Un programa Java lanza a otro con argumentos
8 · Tuberías	Lector, Filtro y Ordenador encadenados
9 · Ejercicio_Pipeline	El pendiente: mismo patrón que el 8
10 · Tarea1-PSP	Gestor con menú (simulado)

PARTE 0

Lo que necesitas saber antes de nada

Si lo entiendes, el resto es repetir el mismo patrón.

1. ¿Qué es un proceso?

Un **proceso** es un programa en ejecución. Cuando abres el Bloc de notas, el sistema crea un proceso. Tu programa Java, mientras corre, también es un proceso.

En esta unidad tu programa (el **padre**) arranca otros programas (los **hijos**) y los controla. Es como un jefe que reparte tareas y luego recoge los resultados.

> **PARA QUÉ SIRVE**
> Sirve para reutilizar programas que ya existen (ping, sort...), dividir un trabajo en piezas independientes, y que si una pieza falla no se caiga todo.

2. Los tres canales de todo proceso

Cada proceso tiene tres "tuberías" por las que entra y sale información:

Canal	Qué es	Desde Java
stdin (entrada)	Lo que el proceso recibe. Normalmente, lo que teclea el usuario.	`getOutputStream()`
stdout (salida)	Lo que escribe normalmente, p. ej. con println.	`getInputStream()`
stderr (errores)	Lo que escribe cuando algo va mal.	`getErrorStream()`

OJO
Los nombres confunden. `getInputStream()` es la **salida** del hijo, porque es la que tú **lees** (entra a tu programa). `getOutputStream()` es la **entrada** del hijo, porque es donde tú **escribes** (sale de tu programa). Mira siempre desde el punto de vista del padre.

3. El código de salida

Cuando un proceso termina, deja un número: su **código de salida**. Por convenio: **0 = todo bien**, cualquier otro número = algo falló. El padre lo recoge con `waitFor()`. Los hijos lo fijan con `System.exit(n)`. Si no llaman a exit y terminan con normalidad, el código es 0.

4. La receta de 4 pasos

Casi todos los ejercicios son esto:

```
// 1) PREPARAR el comando (aún no se ejecuta)
ProcessBuilder pb = new ProcessBuilder(
        "ping", "-c", "4", "127.0.0.1");

// 2) CONFIGURAR (redirecciones...) ANTES de lanzar
pb.redirectOutput(new File("salida.txt"));

// 3) LANZAR
Process p = pb.start();

// 4) ESPERAR y recoger el código de salida
int codigo = p.waitFor();
```

EXAMEN
¿Qué diferencia hay entre `ProcessBuilder` y `Process`?
`ProcessBuilder` **prepara y configura** el comando. `Process` es el proceso **ya en marcha**, el "mando a distancia" para leerlo, esperarlo o matarlo.

PARTE 0 · CAJA DE HERRAMIENTAS

Qué hace cada método

Para consultarla cuando dudes de una pieza.

Preparar y lanzar

Método / cosa	Para qué sirve
`new ProcessBuilder("ping", "8.8.8.8")`	Prepara el comando. Cada palabra va en un argumento **separado**.
`new ProcessBuilder(lista)`	Igual, pero con una `List<String>`. Útil cuando el comando cambia de tamaño.
`pb.start()`	Lanza el proceso y devuelve un `Process`. Lanza `IOException` si el comando no existe.
`pb.command()`	Devuelve la lista del comando (se puede modificar antes de lanzar).

Canales (leer y escribir)

Método / cosa	Para qué sirve
`p.getInputStream()`	Leer lo que el hijo escribe por **stdout**.
`p.getErrorStream()`	Leer lo que el hijo escribe por **stderr**.
`p.getOutputStream()`	Escribir en el **stdin** del hijo (hay que cerrarlo al acabar).

Redirecciones (antes de start)

Método / cosa	Para qué sirve
`pb.inheritIO()`	El hijo usa tu consola: lo que escribe sale por tu pantalla.
`pb.redirectOutput(File)`	stdout a un fichero. **Borra** lo que hubiera.
`Redirect.appendTo(File)`	stdout a un fichero **añadiendo** al final: `pb.redirectOutput(Redirect.appendTo(f))`.
`pb.redirectError(...)`	Lo mismo, pero para stderr.
`pb.redirectErrorStream(true)`	Mezcla stderr dentro de stdout (un solo canal).
`pb.redirectInput(File)`	El hijo lee su entrada de un fichero.
`Redirect.INHERIT`	Usar el canal del padre (como inheritIO pero para uno solo).

Esperar, vigilar y matar

Método / cosa	Para qué sirve
`p.waitFor()`	Espera sin límite a que termine. Devuelve el código de salida.
`p.waitFor(5, TimeUnit.SECONDS)`	Espera como máximo 5 s. Devuelve `true` si terminó, `false` si no. **No lo mata.**
`p.exitValue()`	Código de salida, pero solo si ya terminó (si no, lanza excepción).
`p.isAlive()`	¿Sigue vivo? `true`/`false`.
`p.pid()`	Número identificador del proceso en el sistema.
`p.destroy()`	Petición **educada** de terminar. El proceso puede tardar o ignorarla.
`p.destroyForcibly()`	Lo mata **a la fuerza**, sin opción a negarse.

Método / cosa	Para qué sirve
`p.onExit()`	Devuelve una promesa que se cumple al terminar; con `thenAccept` ejecutas algo **sin bloquear**.

Tuberías

Método / cosa	Para qué sirve
`ProcessBuilder.startPipeline(lista)`	Lanza varios procesos **conectados**: la salida de cada uno es la entrada del siguiente. Devuelve `List<Process>`.

Datos del sistema

Método / cosa	Para qué sirve
`System.getProperty("os.name")`	Nombre del sistema operativo ("Windows 11", "Linux"...).
`System.getProperty("java.home")`	Carpeta donde está instalado el Java que usas.
`System.getProperty("java.class.path")`	Dónde están tus clases. El hijo lo necesita para encontrar su clase.
`File.separator`	El separador de carpetas del SO (`\` o `/`).
`System.exit(n)`	Termina el programa ya, con código `n`.

Leer y escribir texto

Método / cosa	Para qué sirve
`BufferedReader.readLine()`	Lee una línea. Devuelve `null` cuando ya no hay más.
`InputStreamReader`	Traduce bytes a caracteres. Va entre el stream y el BufferedReader.
`PrintWriter.println()`	Escribe una línea. Con autoflush `true` se envía al momento.

Método / cosa	Para qué sirve
`flush()` / `close()`	`flush` envía lo pendiente; `close` además cierra el canal (y avisa de fin de datos).
`new FileWriter(f, true)`	El `true` activa el modo **añadir** (append).
`StandardCharsets.UTF_8`	Codificación que respeta tildes y ñ al pasar texto entre procesos.
`Files.newBufferedReader(Path)`	Abre un fichero para leerlo línea a línea.

1 · ProgramacionMultiprocesos

LectorProceso

Lanzar un comando y leer lo que escribe.

PARA QUÉ SIRVE
Tu programa ejecuta `dir` (Windows) o `ls -la` (Linux), **captura el texto** que el comando imprime y lo muestra con un prefijo. Es la base de todo lo demás: lanzar un proceso y hablar con él.

BLOQUE 1 Elegir el comando según el SO

Los comandos cambian entre Windows y Linux. `System.getProperty("os.name")` devuelve el nombre del SO; con `toLowerCase().contains("win")` sabes si es Windows.

En Windows `dir` no es un programa, es un comando interno de `cmd`, por eso se llama como `cmd.exe /c dir` (`/c` = ejecuta y cierra).

```
boolean win = System.getProperty("os.name")
        .toLowerCase().contains("win");
List<String> cmd = win
    ? List.of("cmd.exe", "/c", "dir")
    : List.of("ls", "-la");
```

BLOQUE 2 Preparar y lanzar

`new ProcessBuilder(cmd)` solo prepara. `pb.start()` crea el hijo y te da el `Process`.

```
ProcessBuilder pb = new ProcessBuilder(cmd);
Process proceso = pb.start();
```

BLOQUE 3 Leer la salida línea a línea

`getInputStream()` es lo que el hijo escribe. Pero son bytes, así que se envuelve: `InputStreamReader` (bytes a texto) y `BufferedReader` (permite `readLine()`).

`readLine()` devuelve `null` cuando el hijo ya no escribe más: esa es la condición de parada. El `try (...)` cierra el lector solo al terminar.

```
try (BufferedReader r = new BufferedReader(
        new InputStreamReader(proceso.getInputStream()))) {
    String linea;
    while ((linea = r.readLine()) != null) {
        System.out.println("[SALIDA HIJO] " + linea);
    }
}
```

BLOQUE 4 Esperar y mirar el código de salida

`waitFor()` bloquea hasta que el hijo acaba y devuelve su código. 0 = éxito.

```
int codigoSalida = proceso.waitFor();
```

OJO

Se lee la salida **antes** de `waitFor()`. Si el hijo escribe mucho y nadie lee, su buffer se llena y se queda bloqueado esperando; tú esperando a que acabe: **bloqueo mutuo**.

EXAMEN

¿Por qué `getInputStream()` y no `getOutputStream()` para leer?

Porque el nombre es desde el punto de vista del padre: lo que el hijo escribe **entra** a tu programa.

2 · ProgramacionMultiprocesos

RedireccionProceso

Decidir adónde va la salida sin leerla tú.

PARA QUÉ SIRVE
Si solo quieres que la salida aparezca por pantalla o se guarde en un fichero, **no hace falta** leerla línea a línea. Se la redirige antes de lanzar y Java lo hace por ti.

BLOQUE 1 Heredar la consola

`inheritIO()` hace que stdin, stdout y stderr del hijo sean los tuyos: lo que el hijo escribe sale en tu pantalla.

```
ProcessBuilder pb = new ProcessBuilder("ping", "127.0.0.1");
pb.inheritIO();
```

BLOQUE 2 Alternativa: mandarlo a un fichero (comentada en el zip)

En su lugar podrías usar `redirectOutput(File)`. Una cosa **o** la otra, no hacen falta las dos.

```
File log = new File("salida_ping.txt");
pb.redirectOutput(log);
```

BLOQUE 3 Lanzar y esperar

Sin el `waitFor()` el programa padre podría terminar antes que el hijo.

```
Process p = pb.start();
p.waitFor();
```

Las opciones, comparadas

Opción	Efecto
`inheritIO()`	A tu consola
`redirectOutput(File)`	A un fichero. **Sobrescribe**.
`redirectOutput(Redirect.appendTo(f))`	A un fichero. **Añade** al final.
`redirectError(File)`	Los errores, a otro fichero.
`redirectErrorStream(true)`	Errores mezclados con la salida normal.

OJO

Siempre se configura **antes** de `start()`. Después ya no tiene efecto.

En Linux, `ping 127.0.0.1` sin `-c` **no termina nunca**, y tu `waitFor()` se quedaría esperando. En Windows hace 4 paquetes y acaba. Por eso en otros proyectos se usa `-n 4` / `-c 4`.

EXAMEN

Si redirijo la salida a un fichero, ¿qué lee `getInputStream()`?

Nada: ya no hay salida que leer porque se desvió al fichero.

3 · ProgramacionMultiprocesos/ejercicios

Ejercicio1

Auditoría de red: ping guardado en fichero.

PARA QUÉ SIRVE
Aplica la redirección: ejecuta `ping 8.8.8.8`, guarda todo en `resultado_ping2.txt` y avisa por consola del resultado según el código de salida.

El archivo tiene **dos formas** de hacer lo mismo. La A está comentada; la B es la que se ejecuta.

BLOQUE 1 Forma A: todo dentro del main (comentada)

Prepara el ping, redirige a un `File`, lanza, espera y decide con un `if` según el código.

`catch (IOException | InterruptedException e)` captura dos tipos de error en un solo bloque.

```
ProcessBuilder pb = new ProcessBuilder("ping", "8.8.8.8");
pb.redirectOutput(new File("resultado_ping.txt"));
Process process = pb.start();
int codigoSalida = process.waitFor();
if (codigoSalida == 0) { /* éxito */ }
else { /* falló con ese código */ }
```

BLOQUE 2 Forma B: en un método aparte

`hacerPingYGuardar(ip, nombreArchivo)` contiene la lógica y devuelve un **código propio**: `0` = OK, `1` = el ping falló, `-1` = no se pudo ni lanzar el proceso.

Ventajas: el `main` queda limpio, y el método se puede reutilizar con otras IP y ficheros.

```
public static int hacerPingYGuardar(String ip, String nombre){
    ProcessBuilder pb = new ProcessBuilder("ping", ip);
    pb.redirectOutput(new File(nombre));
    try {
        int c = pb.start().waitFor();
        return (c == 0) ? 0 : 1;
    } catch (IOException | InterruptedException e) {
        return -1;
    }
}
```

BLOQUE 3 Traducir el código a un mensaje

El `switch` con flechas (`case 0 ->`) es la forma moderna: sin `break` y más corta.

```
switch (resultado) {
    case 0 -> System.out.println("¡Exito!");
    case 1 -> System.out.println("Error: no se contactó");
    case -1 -> System.out.println("Error del sistema");
}
```

OJO

`redirectOutput(File)` **sobrescribe** el fichero cada vez. Si lo quieres acumulado, usa `appendTo`.

El `resultado_ping2.txt` del zip es salida real de Windows. Los símbolos raros (`&#xFFFD;`) no son un fallo tuyo: es la codificación de la consola de Windows.

EXAMEN

¿Qué distingue `IOException` de `InterruptedException` aquí?

`IOException`: no se pudo lanzar el comando o escribir el fichero. `InterruptedException`: alguien interrumpió el hilo mientras esperaba en `waitFor()`.

4 · ProgramacionMultiprocesos

ControlCicloVidaProceso

No esperar eternamente: poner un límite y matar.

PARA QUÉ SIRVE
Si un proceso se cuelga, `waitFor()` te deja esperando para siempre. Aquí pones un **tiempo máximo**; si no acaba, primero se lo pides con educación y luego lo matas por la fuerza.

BLOQUE 1 Un proceso que dura 10 segundos

`sleep 10` en Linux y `timeout /t 10 /nobreak` en Windows simulan un trabajo largo. Se obtiene el `pid()`.

```
ProcessBuilder pb = esWindows
    ? new ProcessBuilder("cmd.exe", "/c",
                         "timeout /t 10 /nobreak")
    : new ProcessBuilder("sleep", "10");
Process proceso = pb.start();
long pid = proceso.pid();
```

BLOQUE 2 Aviso asíncrono con onExit()

`onExit()` devuelve una promesa (`CompletableFuture`). Con `thenAccept(...)` dices **qué hacer cuando el proceso termine**, sin bloquear: el programa sigue ejecutándose mientras tanto.

```
proceso.onExit().thenAccept(p ->
    System.out.println("Terminó PID " + p.pid()));
```

BLOQUE 3 Esperar con límite

`waitFor(3, TimeUnit.SECONDS)` espera como mucho 3 s. Devuelve `true` si terminó y `false` si no. Como el proceso dura 10 s, devolverá `false`.

```
boolean termino = proceso.waitFor(3, TimeUnit.SECONDS);
```

BLOQUE 4 **Si no terminó: destroy y, si hace falta, destroyForcibly**

Primero `destroy()` (petición educada). Se le dan 2 s más. Si sigue vivo, `destroyForcibly()` (a la fuerza).

```
if (!termino) {
    proceso.destroy();
    if (!proceso.waitFor(2, TimeUnit.SECONDS)) {
        proceso.destroyForcibly();
    }
}
```

BLOQUE 5 **Resumen final**

Un último `waitFor()` recoge el código. Debe cumplirse `isAlive() == false`.

```
int codigoSalida = proceso.waitFor();
proceso.isAlive();   // false
```

destroy vs destroyForcibly

Método	Qué hace
`destroy()`	Pide terminar (en Linux, señal SIGTERM). El proceso puede limpiar y cerrarse. Código típico: **143**.
`destroyForcibly()`	Mata sin preguntar (SIGKILL). Código típico: **137**.

OJO

`waitFor(timeout)` **no mata** al proceso: solo deja de esperar. Si no llamas a `destroy()`, sigue corriendo.

Un proceso matado **no devuelve 0**: no terminó su trabajo con normalidad.

En el `catch (InterruptedException)` se restaura la marca con `Thread.currentThread().interrupt()`: buena práctica.

EXAMEN

Esquema de 'espera con límite'

`waitFor(n)` → si `false` → `destroy()` → `waitFor(m)` → si `false` → `destroyForcibly()` → `waitFor()` final.

5 · ProgramacionMultiprocesos

InteraccionProceso

Mandar datos al hijo (su entrada).

PARA QUÉ SIRVE
Hasta ahora solo **leías** del hijo. Aquí además le **escribes** datos, como si alguien los tecleara. Sirve para programas que esperan entrada: `sort`, un intérprete, un filtro...

Esquema:

```
TU PROGRAMA --getOutputStream()--> stdin  del HIJO
TU PROGRAMA <--getInputStream()--- stdout del HIJO
```

BLOQUE 1 Lanzar un hijo que lee una línea

Se lanza Python con un mini-programa: lee **una** línea de su entrada y responde "Hola " + esa línea. `-u` = salida sin buffer; `-c` = ejecuta el código que sigue.

```
ProcessBuilder pb = new ProcessBuilder("python", "-u", "-c",
  "import sys; n=sys.stdin.readline(); print('Hola '+n)");
Process p = pb.start();
```

BLOQUE 2 Escribir en la entrada del hijo

`getOutputStream()` + `PrintWriter`. `println` escribe la línea y `flush()` fuerza el envío.

El `try (...)` **cierra** el writer al salir. Cerrar el canal equivale a decirle al hijo "ya no hay más datos".

```
try (PrintWriter w = new PrintWriter(p.getOutputStream())) {
    w.println("Carlos");
    w.flush();
}
```

BLOQUE 3 **Leer la respuesta**

Se lee su salida con un `Scanner` sobre `getInputStream()`. `hasNextLine()` comprueba que haya algo antes de leer.

```
try (Scanner sc = new Scanner(p.getInputStream())) {
    if (sc.hasNextLine())
        System.out.println("Respuesta: " + sc.nextLine());
}
p.waitFor();
```

OJO

Cierra siempre el canal de escritura. Si el hijo lee hasta el fin de entrada (como `sort`), no empieza a trabajar hasta que cierras, y el programa se queda **colgado**. En este ejemplo el hijo solo lee una línea, por eso aguantaría sin cerrar, pero cerrar es la costumbre correcta.

Necesita Python instalado. En Linux/Mac el comando suele ser `python3`.

Aviso: en tu zip original ese texto de Python decía `sys.stdn` (con una errata: falta la i). Así Python fallaría con un error. En la copia comentada lo escribí bien como `sys.stdin`.

EXAMEN

¿Qué pasa si no cierro el `PrintWriter` y el hijo es `sort`?

`sort` espera más líneas hasta recibir fin de entrada. Sin cerrar, nunca lo recibe: bloqueo.

6 · Diagnostico de red

Main (ping + log)

ping según el SO, guardado en un log con cabecera.

PARA QUÉ SIRVE
Programa que hace un `ping` a `127.0.0.1` y guarda todo en `registro_ping.log`, **sin perder** las ejecuciones anteriores. Cada ejecución lleva una cabecera con fecha y hora. Es la base del ejercicio de auditoría.

BLOQUE 1 Fichero de log y cabecera

`escribirCabeceraLog` abre el fichero en modo **añadir** (`new FileWriter(file, true)`) y escribe una cabecera con la fecha.

`DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss")` define cómo se ve la fecha; `LocalDateTime.now().format(...)` la aplica.

```
DateTimeFormatter f = DateTimeFormatter
        .ofPattern("yyyy/MM/dd HH:mm:ss");
String ts = LocalDateTime.now().format(f);
try (PrintWriter w =
        new PrintWriter(new FileWriter(file, true))) {
    w.println("EJECUCIÓN: " + ts);
}
```

BLOQUE 2 Comando según el SO

En Windows el número de paquetes es `-n`; en Linux, `-c`. Con 4 paquetes el ping termina solo.

```
if (os.contains("win"))
    pb = new ProcessBuilder("ping","-n","4","127.0.0.1");
else
    pb = new ProcessBuilder("ping","-c","4","127.0.0.1");
```

BLOQUE 3 Redirigir salida y errores al log (añadiendo)

Con `appendTo` la salida del ping se **suma** a la cabecera. Sin él, el fichero se sobrescribiría y la cabecera desaparecería.

```
pb.redirectOutput(Redirect.appendTo(logFile));
pb.redirectError(Redirect.appendTo(logFile));
```

BLOQUE 4 Lanzar y mostrar datos del proceso

`pid()` = su identificador. `isAlive()` justo tras lanzar suele ser `true`; tras `waitFor()` será `false`.

El código de salida se traduce a texto: `exitCode == 0 ? "OK" : "ERROR"`.

```
Process proceso = pb.start();
proceso.pid();      // PID
proceso.isAlive();  // true
int exitCode = proceso.waitFor();
proceso.isAlive();  // false
```

OJO

El orden importa: primero la cabecera, luego el ping. Y los dos en modo añadir.

En `catch (InterruptedException)` se restaura con `Thread.currentThread().interrupt()`.

EXAMEN

¿Por qué `Redirect.appendTo` y no `redirectOutput(File)`?

Porque este último borra el contenido previo, incluida la cabecera y las ejecuciones anteriores.

7 · Diagnostico de red/lanzador

Lanzador y Registrador

Un programa Java lanza a otro y le pasa un dato.

PARA QUÉ SIRVE
Son **dos programas Java**. `Lanzador` (padre) arranca a `Registrador` (hijo) pasándole un nombre de usuario. El hijo apunta ese usuario con la hora en `log.txt` y devuelve un **código de salida**.

```
Lanzador --"Jander"--> Registrador --> log.txt
   ^                        |
   +---- código de salida --+
```

A) Lanzador (el padre)

BLOQUE 1 Localizar java y las clases

Para lanzar otro programa Java hay que ejecutar el comando `java`. `java.home` + `bin` + `java` da su ruta; `File.separator` usa la barra correcta del SO.

`java.class.path` dice dónde están **tus clases compiladas**. Sin él el hijo no encuentra su clase (`ClassNotFoundException`).

```
String java = System.getProperty("java.home")
    + File.separator + "bin" + File.separator + "java";
String cp = System.getProperty("java.class.path");
```

BLOQUE 2 Montar el comando y lanzar

Equivale a escribir en la consola `java -cp <cp> lanzador.Registrador Jander`. La clase lleva su paquete (`lanzador.Registrador`). Lo último, `usuarioAEnviar`, llegará al hijo como `args[0]`.

`inheritIO()` hace que lo que escriba el hijo salga por la consola del padre.

```
ProcessBuilder pb = new ProcessBuilder(java, "-cp", cp,
        "lanzador.Registrador", usuarioAEnviar);
pb.inheritIO();
Process process = pb.start();
int outCode = process.waitFor();
```

B) Registrador (el hijo)

BLOQUE 3 **Validar el argumento**

`args` son los textos que le pasa el padre. Si no hay ninguno, no puede trabajar: escribe el error por `System.err` y sale con `System.exit(1)`.

```
if (args.length < 1) {
    System.err.println("Falta el nombre de usuario");
    System.exit(1);
}
String user = args[0];
```

BLOQUE 4 **Apuntar en el log**

Abre `log.txt` en modo **añadir** y escribe `usuario accedió el <fecha>`. Si falla la escritura avisa por `System.err`.

```
try (PrintWriter out =
        new PrintWriter(new FileWriter("log.txt", true))) {
    out.println(user + " accedió el "
        + LocalDateTime.now());
}
```

Códigos de salida del Registrador

Código	Significa
`0`	Todo bien: terminó sin llamar a `exit`
`1`	Faltó el argumento

OJO
El `log.txt` del zip tiene 3 líneas porque se ejecutó varias veces: cada una **añade** una. Si usaras `new FileWriter(nombre)` sin `true`, solo quedaría la última.

EXAMEN
¿Por qué el hijo comprueba `args.length`, si ya se lo manda el padre?

Porque cada programa puede ejecutarse solo. **Quien recibe el dato, lo valida.** Sin comprobarlo daría `ArrayIndexOutOfBoundsException`.

8 · Procesamiento_de_datos_en_tuberia

Tubería de 3 procesos

El patrón más importante del examen.

PARA QUÉ SIRVE
Una **tubería** encadena procesos: la salida de uno es la entrada del siguiente, como una cadena de montaje. Es el `|` de la consola (`cat | grep | sort`), pero hecho con 3 programas Java.

```
alumnos.txt
   |
[LectorProceso]     lee el fichero, lo suelta
   |
[FiltroAsignatura]  solo una asignatura y nota >= 5
   |
[OrdenadorAlumnos]  ordena por nombre
   |
psp_aprobados_ordenados.txt
```

Cada línea de `alumnos.txt` tiene el formato `Nombre,ASIGNATURA,nota`, por ejemplo `Ana López,AD,9.5`.

1) LectorProceso

Para qué sirve: es el primer eslabón. Lee el fichero que recibe en `args[0]` y lo escribe por stdout.

BLOQUE 1 Argumento y apertura del fichero

Sin argumento sale con código 1. `Path.of` crea la ruta y `Files.newBufferedReader(..., UTF_8)` abre el fichero.

```
Path fichero = Path.of(args[0]);
BufferedReader lector =
    Files.newBufferedReader(fichero, StandardCharsets.UTF_8);
```

BLOQUE 2 Escribir por stdout en UTF-8

Se envuelve `System.out` en un `PrintWriter` con UTF-8, para que las tildes (Pérez, López) no se estropeen al viajar entre procesos. El `true` es **autoflush**: envía en cada `println`.

Los errores van por `System.err` (otro canal) para no mezclarse con los datos de la tubería. Código 1 = mal uso, 2 = no se pudo leer.

```
PrintWriter salida = new PrintWriter(
  new OutputStreamWriter(System.out, StandardCharsets.UTF_8),
  true);
while ((linea = lector.readLine()) != null)
    salida.println(linea);
```

2) FiltroAsignatura

Para qué sirve: lee por stdin y deja pasar **solo** los alumnos de una asignatura con nota **≥ 5**. La asignatura llega por `args[0]`.

BLOQUE 1 Leer todo el stdin

Lee con `readLine()` hasta `null`. Se salta las líneas vacías (`isBlank()`).

```
while ((linea = entrada.readLine()) != null) {
    if (linea.isBlank()) continue;
    ...
}
```

BLOQUE 2 Trocear y comprobar

`split(",")` separa por comas en un array: `[nombre, asignatura, nota]`. Si hay menos de 3 campos, la línea está mal y se ignora.

`equalsIgnoreCase` compara sin importar mayúsculas.

```
String[] c = linea.split(",");
if (c.length < 3) continue;
if (!c[1].trim().equalsIgnoreCase(buscada)) continue;
```

BLOQUE 3 Convertir la nota y filtrar

`Double.parseDouble` pasa el texto a número; si no es un número lanza `NumberFormatException` y se ignora la línea. `NOTA_MINIMA` es una constante (`static final`) con valor 5.0.

```
double nota;
try { nota = Double.parseDouble(c[2].trim()); }
catch (NumberFormatException e) { continue; }
if (nota >= NOTA_MINIMA)
    salida.println(c[0].trim()+","+c[1].trim()+","+nota);
```

> **OJO**
> Hay que leer **toda** la entrada aunque no te interese. Si el filtro dejara de leer, el proceso anterior se bloquearía al intentar escribir en una tubería llena.

3) OrdenadorAlumnos

Para qué sirve: lee todas las líneas, las **ordena por nombre** y las escribe. Es el equivalente a `sort`.

BLOQUE 1 Leer TODO antes de escribir

Para ordenar necesitas tener todos los datos en memoria: no se puede ordenar sobre la marcha. Se guardan en un `ArrayList`.

```
List<String> registros = new ArrayList<>();
while ((linea = entrada.readLine()) != null)
    if (!linea.isBlank()) registros.add(linea);
```

BLOQUE 2 Ordenar con un comparador

`Comparator.comparing(OrdenadorAlumnos::nombre)` dice "ordena usando lo que devuelve `nombre()`". `OrdenadorAlumnos::nombre` es una referencia al método auxiliar de abajo.

`nombre()` saca lo que hay antes de la primera coma con `indexOf(',')` y `substring(0, coma)`.

```
registros.sort(
    Comparator.comparing(OrdenadorAlumnos::nombre));

private static String nombre(String reg) {
    int coma = reg.indexOf(',');
    return coma >= 0 ? reg.substring(0, coma).trim()
                     : reg.trim();
}
```

EXAMEN
Cómo ordenar por nota de mayor a menor

Cambia el comparador por `Comparator.comparingDouble(OrdenadorAlumnos::nota).reversed()` y crea un método `nota()` que haga `Double.parseDouble(reg.split(",")[2].trim())`.

4) TuberiaTresProcesos (el montador)

Para qué sirve: es el `main` que prepara los tres procesos, los conecta y espera el resultado.

BLOQUE 1 Parámetros con valor por defecto

El operador ternario `cond ? a : b` elige: si hay argumento lo usa; si no, un valor por defecto.

```
String ruta = args.length > 0 ? args[0] : "alumnos.txt";
String asig = args.length > 1 ? args[1] : "PSP";
```

BLOQUE 2 Preparar los tres ProcessBuilder

Cada uno lanza una clase con `java -cp`. Todavía no se ejecutan.

```
ProcessBuilder lector = new ProcessBuilder(javaBin, "-cp",
    cp, "org.example.LectorProceso", ruta);
ProcessBuilder filtro = new ProcessBuilder(javaBin, "-cp",
    cp, "org.example.FiltroAsignatura", asig);
ProcessBuilder ordenador = new ProcessBuilder(javaBin,
    "-cp", cp, "org.example.OrdenadorAlumnos");
```

BLOQUE 3 Errores a consola y salida final a fichero

`redirectError(INHERIT)` en los tres: los errores salen por tu pantalla. Los datos (stdout) **no se tocan**: van por la tubería.

Solo el **último** redirige su salida a un fichero, con nombre `psp_aprobados_ordenados.txt`, `ad_...`, etc.

```
lector.redirectError(Redirect.INHERIT);   // y los otros 2
File salida = new File(
    asig.toLowerCase() + "_aprobados_ordenados.txt");
ordenador.redirectOutput(salida);
```

BLOQUE 4 Lanzar la tubería y esperar al último

`startPipeline` arranca los tres a la vez y conecta cada salida con la siguiente entrada. Solo se espera al último: cuando él acaba, ya han acabado los demás.

```
List<Process> ps = ProcessBuilder.startPipeline(
        List.of(lector, filtro, ordenador));
int codigo = ps.get(ps.size() - 1).waitFor();
```

Resultados que trae el zip

Fichero	Contenido
`psp_aprobados_ordenados.txt`	Beatriz Alonso 9.2 · Elena Navarro 5.0 · Gonzalo Ramírez 8.1 · Juan Pérez 7.5 · Lucía Fernández 6.0
`ad_aprobados_ordenados.txt`	Ana López 9.5 · María Gómez 8.0 · Sofía Domínguez 5.5
`sge_aprobados_ordenados.txt`	David Sánchez 6.5

Se quedan fuera los suspensos: Carlos Ruíz (4.2), Javier Torres (4.9) y Pedro Martín (3.5). Están ordenados por **nombre**, no por nota.

OJO

Los procesos del **medio** no se pueden redirigir: solo la entrada del primero y la salida del último. Si lo intentas, `startPipeline` lanza una excepción.

Si la tubería necesita datos de entrada para el primero, se puede usar `redirectInput` (solo en el primero).

EXAMEN

¿Por qué solo `waitFor()` del último?

Porque necesita todos los datos de los anteriores para terminar: cuando acaba él, ya acabaron todos.

9 · Ejercicio_Pipeline

El pendiente

Mismo patrón que la tubería, con otros datos.

PARA QUÉ SIRVE
El `Main` está vacío (`cargarDatos()` sin cuerpo). Hay que construir una tubería como la del punto 8 con el fichero `datos_brutos.txt`. Es el candidato a salir en el examen.

Cada línea del fichero:

```
ID: 104 | Usuario: mgarcia | Fecha: 15/03/2023 | Estado: OK
```

Objetivo: quedarse con las líneas `Estado: OK`, ordenarlas por ID de menor a mayor y guardarlas en `ok_ordenados.txt`.

Las piezas (también en los archivos de solución)

Clase	Qué hace
`LectorDatos`	Lee `src/main/resources/datos_brutos.txt` y lo escribe por stdout.
`FiltroOK`	Deja pasar solo las líneas que contengan `"Estado: OK"`.
`OrdenadorPorId`	Lee todo, ordena por el número del ID y escribe.
`Main`	Monta los 3 `ProcessBuilder`, redirige el último a `ok_ordenados.txt` y usa `startPipeline`.

BLOQUE 1 Sacar el ID de una línea

`split("\\|")` separa por el carácter `|`. Se escribe con **dos barras** porque `|` tiene significado especial en las expresiones regulares y hay que escaparlo.

`replace("ID:", "")` quita la etiqueta, `trim()` los espacios y `Integer.parseInt` lo pasa a número.

```
private static int id(String linea) {
    String parte = linea.split("\\|")[0];   // "ID: 104 "
    return Integer.parseInt(
        parte.replace("ID:", "").trim());
}
```

BLOQUE 2 **Ordenar por número**

`comparingInt` compara **números**. Si compararas el texto, `"100"` iría antes que `"99"`.

```
lineas.sort(Comparator.comparingInt(OrdenadorPorId::id));
```

BLOQUE 3 **Ruta del fichero**

Al ejecutar desde el IDE, la carpeta de trabajo es la raíz del proyecto, así que la ruta relativa es `src/main/resources/datos_brutos.txt`.

```
new ProcessBuilder(java, "-cp", cp,
  "org.example.LectorDatos",
  "src/main/resources/datos_brutos.txt");
```

Resultado esperado
IDs 100, 101, 103, 104 y 106 en ese orden. Se descartan el 102 y el 105, que son ERROR.

EXAMEN
Variantes habituales

Filtrar por `Estado: ERROR`. Orden descendente con `.reversed()`. Ordenar por fecha (`LocalDate.parse(txt, DateTimeFormatter.ofPattern("dd/MM/yyyy"))`). Contar líneas en vez de ordenar.

10 · Tarea1-PSP

Gestor de procesos (simulado)

Menú con Scanner y una lista.

PARA QUÉ SIRVE
Un menú de consola para "crear", "listar" y "terminar" procesos. **Ojo: es una simulación.** No lanza procesos reales: cada "proceso" es solo un texto guardado en un `ArrayList`. Practica el menú, el `Scanner` y la lógica de buscar y cambiar elementos.

BLOQUE 1 La lista y el bucle del menú

`procesos` es una lista `static` (compartida por todos los métodos). El `do-while` repite el menú hasta que la opción sea 4.

```
static ArrayList<String> procesos = new ArrayList();
do {
    mostrarMenu();
    ...
} while (opcion != 4);
```

BLOQUE 2 Leer la opción sin romperse

`hasNextInt()` comprueba que lo escrito sea un número **antes** de leerlo. `nextInt()` deja el Enter sin leer, así que se limpia con `nextLine()` (si no, el siguiente `nextLine()` leería vacío).

Si no era un número, `input.next()` **descarta** ese texto para no entrar en un bucle infinito.

```
if (input.hasNextInt()) {
    opcion = input.nextInt();
    input.nextLine();       // consume el salto pendiente
    ...
} else {
    input.next();           // descarta lo no numérico
    opcion = 0;
}
```

BLOQUE 3 Crear y listar

Crear añade `"codigo | Estado: VIVO"` a la lista. Listar comprueba `isEmpty()` y recorre con un `for-each`.

```
procesos.add(codigo + " | Estado: VIVO");

for (String proceso : procesos)
    System.out.println("PID:" + proceso);
```

BLOQUE 4 Terminar por código

Recorre la lista con índice. `startsWith(codigo + " |")` evita confundir el código `1` con el `10`. Al encontrarlo, `set(i, ...)` reemplaza el elemento por la versión TERMINADO.

```
for (int i = 0; i < procesos.size(); i++) {
    if (procesos.get(i).startsWith(codigo + " |")) {
        procesos.set(i, codigo + " | Estado: TERMINADO");
        return;
    }
}
```

Cómo sería la versión REAL

Guardar objetos `Process` en un `Map<Long, Process>` con su `pid()` como clave. Buscar por PID es directo y tienes el objeto para controlarlo.

```
static Map<Long, Process> procs = new HashMap<>();

// crear
Process p = new ProcessBuilder("sleep", "300").start();
procs.put(p.pid(), p);

// listar
procs.forEach((pid, pr) ->
    System.out.println(pid + " vivo=" + pr.isAlive()));

// terminar
procs.get(pid).destroy();
```

OJO
En `crearProceso` se crea un segundo `Scanner` sobre `System.in`. Funciona aquí, pero lo correcto es reutilizar el del `main`.

REPASO

Chuleta y preguntas de examen

Para leer la víspera.

Chuleta en una página

Quiero…	Uso
Lanzar	`new ProcessBuilder(...).start()`
Leer su salida	`getInputStream()` + `BufferedReader.readLine()`
Escribirle	`getOutputStream()` y **cerrar**
Esperar	`waitFor()` → 0 es OK
Con límite	`waitFor(n, TimeUnit)` → `destroy()` → `destroyForcibly()`
Redirigir	`inheritIO`, `redirectOutput`, `appendTo`, `redirectError`
Encadenar	`ProcessBuilder.startPipeline(List.of(...))`
Lanzar clase Java	`java.home` + `-cp java.class.path` + `paquete.Clase` + args
Leer argumentos	`args.length` y `args[0]`; validar y `System.exit(n)`

Patrón de un proceso de tubería

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(System.in, StandardCharsets.UTF_8));
PrintWriter out = new PrintWriter(
    new OutputStreamWriter(System.out,
        StandardCharsets.UTF_8), true);

String linea;
while ((linea = in.readLine()) != null) {
    // procesar y out.println(...)
}
```

Preguntas de teoría con respuesta

¿Diferencia entre proceso e hilo?
Un proceso tiene su propia memoria y es independiente. Los hilos viven dentro de un proceso y comparten su memoria.

¿Qué hace `redirectErrorStream(true)`?
Mezcla stderr con stdout en un solo canal, así solo hay que leer uno.

¿Qué devuelve `waitFor()`?
El código de salida. 0 = éxito; otro valor = error.

¿Por qué `destroy()` no garantiza matar?
Es una petición educada: el proceso puede tardar o ignorarla. Por eso existe `destroyForcibly()`.

¿Por qué leer la salida antes de `waitFor()`?
Si el hijo escribe mucho y nadie lee, su buffer se llena, se bloquea y `waitFor()` no termina nunca.

¿Para qué sirve `java.class.path`?
Para que el hijo Java encuentre sus clases. Sin él: `ClassNotFoundException`.

¿Por qué cerrar el `PrintWriter` hacia el hijo?
Cerrar avisa al hijo de que no hay más datos. Si espera fin de entrada, queda colgado.

¿Se puede redirigir un proceso del medio de la tubería?
No. Solo la entrada del primero y la salida del último.

Errores típicos

Revisa tu código con esta lista

Leer `args[0]` y luego usar el valor escrito a mano. (Leerlo no sirve si no lo pasas al `ProcessBuilder`.)

Usar `redirectOutput(File)` cuando querías añadir: hay que usar `appendTo`.

Olvidar `waitFor()` y que el padre termine antes que el hijo.

Poner el comando en un único texto (`"ping 8.8.8.8"`): cada palabra va en un argumento aparte.

Comparar números como texto al ordenar: usa `comparingInt` o `comparingDouble`.

No incluir el paquete en el nombre de la clase al lanzarla (`org.example.Clase`).